

WHY TWO POINTERS?

Checking all pairs in an array takes $O(N^2)$ time. Two Pointers reduces this to $O(N)$ by leveraging **sorting or structural constraints**.

THE AHA MOMENT

Instead of nested loops `i` and `j` scanning everywhere, we maintain two pointers moving inwards or at different speeds to eliminate invalid search spaces deterministically!

4 CORE VARIANTS

- Opposite Direction:** `'left=0', 'right=N-1'`. Move inwards based on target sum or palindrome.
- Same Direction (Fast/Slow):** `'slow'` tracks write index, `'fast'` scans elements.
- Three Pointers:** Dutch National Flag (`'low', 'mid', 'high'`). Partitioning arrays into 3 groups.
- Merging Arrays:** Two pointers starting at ends of two sorted arrays.

VISUALIZING POINTER MOVEMENT

Opposite Direction (Two Sum II):

```
Left = 0                                Right = N
- 1
  ▼                                    ▼
[ 2,  7, 11, 15 ]   Target = 9
sum = 2 + 15 = 17 > 9 -> Move Right
Leftwards!
sum = 2 + 7  = 9  == 9 -> MATCH FOUND! ✓
```

PREREQUISITES & COMPLEXITY REASONING

Pattern Variant	Time Complexity	Auxiliary Space
Opposite Direction (Sorted)	$O(N)$	$O(1)$
Same Direction (Fast/Slow)	$O(N)$	$O(1)$
Dutch National Flag (3 Pointers)	$O(N)$	$O(1)$
Unsorted Array + Sorting Step	$O(N \log N)$	$O(1)$ or $O(N)$

CLUES THAT SCREAM TWO POINTERS

- "Array is sorted" + "Find pair with sum X" $\rightarrow$ Opposite Pointers
- "Remove duplicates in-place" or "Move elements" $\rightarrow$ Fast/Slow Pointers
- "Sort array of 0s, 1s, and 2s" $\rightarrow$ Dutch National Flag (3 Pointers)
- "Container with most water" / "Trapping rainwater" $\rightarrow$ Two Pointers Trapping

1 REUSABLE POINTER UTILITIES ('PointerUtils')

```
public class PointerUtils {  
    // In-place Swap Helper  
    public static void swap(int[] nums, int i, int j) {  
        int temp = nums[i];  
        nums[i] = nums[j];  
        nums[j] = temp;  
    }  
  
    // Character Swap Helper for char[]  
    public static void swap(char[] s, int i, int j) {  
        char temp = s[i];  
        s[i] = s[j];  
        s[j] = temp;  
    }  
}
```

2 PALINDROME & ALPHANUMERIC HELPERS

```
public class PalindromeUtils {  
    // Validates if char is alphanumeric without Regex overhead  
    public static boolean isAlphaNumeric(char c) {  
        return (c >= 'a' && c <= 'z') ||  
               (c >= 'A' && c <= 'Z') ||  
               (c >= '0' && c <= '9');  
    }  
  
    // Pure 2-Pointer Palindrome Check  
    public static boolean isPalindrome(String s, int left, int right) {  
        while (left < right) {  
            if (s.charAt(left++) != s.charAt(right--)) return false;  
        }  
        return true;  
    }  
}
```

3 REVERSE ARRAY SUBSEGMENT HELPERS

```
public static void reverse(int[] nums, int left, int right) {  
    while (left < right) {  
        int temp = nums[left];  
        nums[left++] = nums[right];  
        nums[right--] = temp;  
    }  
}
```

💡 Used in Rotate Array (LC 189) & Next Permutation (LC 31)!

4 DUPLICATE SKIPPING IN 3SUM / 4SUM

```
// Skip duplicate left elements  
while (left < right && nums[left] == nums[left + 1]) left++;  
// Skip duplicate right elements  
while (left < right && nums[right] == nums[right - 1]) right--;  
left++; right--;
```

Crucial to avoid duplicate triplets in $O(N^2)$ 3Sum search!

PATTERN
1

OPPOSITE DIRECTION POINTERS (Inward Convergence)

e.g. Two Sum II (LC 167), Valid Palindrome (LC 125), Reverse Array

WHEN TO USE

Array is **sorted** or string needs palindrome check.
Finding pairs matching target sum or property.

💡 AHA

If `sum < target`, `left++` increases sum. If `sum > target`, `right--` decreases sum. Guarantees $O(N)$!

TEMPLATE — TWO SUM II (LC 167)

```
public int[] twoSum(int[] numbers, int target) {
    int left = 0, right = numbers.length - 1;
    while (left < right) {
        int sum = numbers[left] + numbers[right];
        if (sum == target) return new int[]{left + 1, right + 1};
        else if (sum < target) left++;
        else right--;
    }
    return new int[]{-1, -1};
}
```

PATTERN
2

SAME DIRECTION POINTERS (Fast / Slow Reader-Writer)

e.g. Remove Duplicates (LC 26), Move Zeroes (LC 283), Remove Element

WHEN TO USE

In-place modification of array/string.
`'slow'` points to next write position; `'fast'` scans input.

TEMPLATE — MOVE ZEROES (LC 283)

```
public void moveZeroes(int[] nums) {
    int slow = 0;
    for (int fast = 0; fast < nums.length; fast++) {
        if (nums[fast] != 0) {
            int temp = nums[slow];
            nums[slow] = nums[fast];
            nums[fast] = temp;
            slow++;
        }
    }
}
```

PATTERN 3DUTCH NATIONAL FLAG (3-Way Partitioning)e.g. Sort Colors (LC 75) — 0s, 1s, 2s	
3 POINTERS MECHANICS <code>`low`</code>: Boundary for 0s (<code>`nums[0..low-1] == 0`</code>) <code>`mid`</code>: Current scanner element <code>`high`</code>: Boundary for 2s (<code>`nums[high+1..N-1] == 2`</code>) When swapping <code>`mid`</code> with <code>`high`</code>, do NOT increment <code>`mid`</code>! Re-examine swapped element.	TEMPLATE — SORT COLORS (LC 75) <pre>public void sortColors(int[] nums) { int low = 0, mid = 0, high = nums.length - 1; while (mid <= high) { if (nums[mid] == 0) { swap(nums, low++, mid++); } else if (nums[mid] == 1) { mid++; } else { swap(nums, mid, high--); // Do NOT mid++! } } }</pre>
PATTERN 4BACKWARDS MERGE POINTERS (Merge Sorted Arrays)e.g. Merge Sorted Array (LC 88)	
WHEN TO USE Merging two sorted arrays in-place when buffer exists at end of array 1. Fill backwards to avoid overwriting!	TEMPLATE — MERGE SORTED ARRAY (LC 88) <pre>public void merge(int[] nums1, int m, int[] nums2, int n) { int p1 = m - 1, p2 = n - 1, p = m + n - 1; while (p2 >= 0) { if (p1 >= 0 && nums1[p1] > nums2[p2]) { nums1[p--] = nums1[p1--]; } else { nums1[p--] = nums2[p2--]; } } }</pre>

PATTERN 5 CONTAINER WITH MOST WATER (Greedy Shrinking) e.g. Container With Most Water (LC 11)

GREEDY CHOICE PROOF

Area = `width \* min(h[l], h[r])`.
Shrinking width reduces potential area. The ONLY way to find a larger area is to move the shorter wall inwards in search of a taller height!

TEMPLATE — LC 11

```
public int maxArea(int[] height) {
    int left = 0, right = height.length - 1, max = 0;
    while (left < right) {
        int w = right - left;
        int h = Math.min(height[left], height[right]);
        max = Math.max(max, w * h);
        if (height[left] < height[right]) left++;
        else right--;
    }
    return max;
}
```

PATTERN 6 TRAPPING RAIN WATER (Max-Left & Max-Right Tracking) e.g. Trapping Rain Water (LC 42)

WHY O(1) SPACE WORKS

Water trapped at index `i` is `min(leftMax, rightMax) - height[i]`.
If `leftMax < rightMax`, the bottleneck is guaranteed to be `leftMax`, so process `left` and `left++`!

TEMPLATE — LC 42

```
public int trap(int[] height) {
    int left = 0, right = height.length - 1;
    int leftMax = 0, rightMax = 0, water = 0;
    while (left < right) {
        if (height[left] < height[right]) {
            if (height[left] >= leftMax) leftMax = height[left];
            else water += leftMax - height[left];
            left++;
        } else {
            if (height[right] >= rightMax) rightMax = height[right];
            else water += rightMax - height[right];
            right--;
        }
    }
    return water;
}
```

PATTERN 7

3SUM / K-SUM GENERALIZATION WITH DUPLICATE PRUNING

e.g. 3Sum (LC 15), 4Sum (LC 18)

K-SUM REDUCTION STRATEGY

- 1. Sort array $O(N \log N)$.
- 2. Fix outer loop element `nums[i]`.
- 3. Skip duplicates: `if (i > 0 && nums[i] == nums[i-1]) continue;`
- 4. Run 2-Pointer inward search on remaining elements!

TEMPLATE — 3SUM (LC 15)

```
public List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue; // Skip dup
        int l = i + 1, r = nums.length - 1, target = -nums[i];
        while (l < r) {
            int sum = nums[l] + nums[r];
            if (sum == target) {
                res.add(Arrays.asList(nums[i], nums[l], nums[r]));
                while (l < r && nums[l] == nums[l + 1]) l++;
                while (l < r && nums[r] == nums[r - 1]) r--;
                l++; r--;
            } else if (sum < target) l++;
            else r--;
        }
    }
    return res;
}
```

TWO POINTERS — WHICH VARIANT TO USE?

Start: What is the array state & goal?

🔍 Sorted Array + Pair Sum Target

→

Opposite Pointers
`left`=0, `right`=N-1`

✂ In-place Removal / Move Zeroes

→

Fast / Slow Pointers
`slow` write, `fast` read

🎯 Partition 3 Categories (0,1,2)

→

Dutch National Flag
`low`, `mid`, `high`

📦 Container / Rainwater Trap

→

Two Pointers & Trapping
Track `leftMax`, `rightMax`

+ 3Sum / 4Sum Triplets

→

Sort + Outer Loop + 2-Pointer
Skip duplicates

🔗 TRIGGER WORDS DIRECTORY

Trigger Word	Variant	Action
"Sorted Array", "Target Sum"	Opposite Pointers	Compare sum to target
"In-place", "Remove elements"	Fast / Slow	Swap `nums[slow]` & `nums[fast]`
"0s, 1s, 2s", "Partition"	Dutch National Flag	Swap with `low` or `high`
"Water trapped", "Container"	Max-Left / Right	Move shorter wall inward

📁 INTERVIEW TRADE-OFFS & FOLLOW-UPS

Interviewer Asks	Your Answer
Can we use HashMap for Two Sum?	Yes, HashMap is $O(N)$ time & $O(N)$ space. Two Pointers achieves $O(N)$ time & $O(1)$ space on sorted arrays!
Why sort first for 3Sum?	Sorting enables duplicate pruning in $O(1)$ time and allows 2-pointer convergence!

EASY — Fundamental Pointer Movements

LC 125 · Valid PalindromeEASY

Pattern: Opposite Pointers
Key Insight: Skip non-alphanumeric chars.

```
public boolean isPalindrome(String s) {
    int l = 0, r = s.length() - 1;
    while (l < r) {
        while (l < r && !Character.isLetterOrDigit(s.charAt(l))) l++;
        while (l < r && !Character.isLetterOrDigit(s.charAt(r))) r--;
        if (Character.toLowerCase(s.charAt(l+)) != Character.toLowerCase(s.charAt(r--))) return false;
    }
    return true;
}
```

MEDIUM — Sorted Pair & Array Merge

LC 167 · Two Sum IIMEDIUM

Pattern: Opposite Pointers
Key Insight: Move left if sum < target, right if >.

```
public int[] twoSum(int[] nums, int target) {
    int l = 0, r = nums.length - 1;
    while (l < r) {
        int sum = nums[l] + nums[r];
        if (sum == target) return new int[]{l + 1, r + 1};
        else if (sum < target) l++;
        else r--;
    }
    return new int[]{-1, -1};
}
```

LC 26 · Remove DuplicatesEASY

Pattern: Fast / Slow Pointers
Key Insight: Copy when `nums[fast] != nums[slow]`.

```
public int removeDuplicates(int[] nums) {
    if (nums.length == 0) return 0;
    int slow = 0;
    for (int fast = 1; fast < nums.length; fast++) {
        if (nums[fast] != nums[slow]) {
            nums[++slow] = nums[fast];
        }
    }
    return slow + 1;
}
```

LC 88 · Merge Sorted ArrayEASY

Pattern: Backwards Merge
Key Insight: Fill from right `m+n-1` to avoid overwrite.

```
public void merge(int[] n1, int m, int[] n2, int n) {
    int p1 = m - 1, p2 = n - 1, p = m + n - 1;
    while (p2 >= 0) {
        if (p1 >= 0 && n1[p1] > n2[p2]) n1[p--] = n1[p1--];
        else n1[p--] = n2[p2--];
    }
}
```

LC 283 · Move ZeroesEASY

Pattern: Fast / Slow Pointers
Key Insight: Swap non-zero at `fast` to `slow`.

```
public void moveZeroes(int[] nums) {
    int slow = 0;
    for (int fast = 0; fast < nums.length; fast++) {
        if (nums[fast] != 0) {
            int tmp = nums[slow]; nums[slow++] = nums[fast]; nums[fast] = tmp;
        }
    }
}
```


MEDIUM — 3Sum, Container & Partition

LC 15 · 3Sum

MEDIUM

Pattern: Sort + Outer Loop + 2-Pointer

Key Insight: Skip duplicate elements in loops.

```
public List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums); List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i-1]) continue;
        int l = i + 1, r = nums.length - 1, target = -nums[i];
        while (l < r) {
            int sum = nums[l] + nums[r];
            if (sum == target) {
                res.add(Arrays.asList(nums[i], nums[l], nums[r]));
                while (l < r && nums[l] == nums[l+1]) l++;
                while (l < r && nums[r] == nums[r-1]) r--;
                l++; r--;
            } else if (sum < target) l++; else r--;
        }
    }
    return res;
}
```

LC 11 · Container With Most Water

MEDIUM

Pattern: Shrinking Pointers

Key Insight: Move shorter wall inward.

```
public int maxArea(int[] h) {
    int l = 0, r = h.length - 1, max = 0;
    while (l < r) {
        max = Math.max(max, (r - l) * Math.min(h[l], h[r]));
        if (h[l] < h[r]) l++; else r--;
    }
    return max;
}
```

HARD — Rainwater & String Compression

LC 75 · Sort Colors

MEDIUM

Pattern: Dutch National Flag

Key Insight: 'low', 'mid', 'high' 3-way swap.

```
public void sortColors(int[] nums) {
    int low = 0, mid = 0, high = nums.length - 1;
    while (mid <= high) {
        if (nums[mid] == 0) swap(nums, low++, mid++);
        else if (nums[mid] == 1) mid++;
        else swap(nums, mid, high--);
    }
}
```

LC 42 · Trapping Rain Water

HARD

Pattern: Max-Left / Right Pointers

Key Insight: Bottleneck is 'min(leftMax, rightMax)'.

```
public int trap(int[] h) {
    int l = 0, r = h.length - 1, lMax = 0, rMax = 0, w = 0;
    while (l < r) {
        if (h[l] < h[r]) {
            if (h[l] >= lMax) lMax = h[l]; else w += lMax - h[l];
            l++;
        } else {
            if (h[r] >= rMax) rMax = h[r]; else w += rMax - h[r];
            r--;
        }
    }
    return w;
}
```

COMPLETE TWO POINTERS PROBLEM LADDER SUMMARY					
LC #	Problem	Pattern	Time	Space	Diff
125	Valid Palindrome	Opposite Pointers	O(N)	O(1)	E
26	Remove Duplicates	Fast / Slow Pointers	O(N)	O(1)	E
283	Move Zeroes	Fast / Slow Pointers	O(N)	O(1)	E
167	Two Sum II	Opposite Pointers	O(N)	O(1)	M
88	Merge Sorted Array	Backwards Merge	O(M+N)	O(1)	E
15	3Sum	Sort + Outer Loop + 2-Pointer	O(N ²)	O(1)	M
11	Container With Most Water	Shrinking Pointers	O(N)	O(1)	M
75	Sort Colors	Dutch National Flag	O(N)	O(1)	M
42	Trapping Rain Water	Max-Left / Right Pointers	O(N)	O(1)	H

DRY RUN — Trapping Rain Water (LC 42)

Height = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]

Step	L, R	h[L], h[R]	LMax, RMax	Trapped Water
1	0, 11	0, 1	0, 0	$h[L]=0 < h[R]=1 \rightarrow LMax=0 \rightarrow L++ (L=1)$
2	1, 11	1, 1	1, 0	$h[R]=1 \leq h[L]=1 \rightarrow RMax=1 \rightarrow R-- (R=10)$
3	2, 10	0, 2	1, 2	$h[L]=0 < h[R]=2 \rightarrow Water += LMax-h[L] = 1-0 = 1! L++$

MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Write Two Sum II in 3 minutes
- ☐ Code Remove Duplicates in-place
- ☐ Implement Move Zeroes with fast/slow
- ☐ Code Dutch National Flag in 5 minutes
- ☐ Write 3Sum with duplicate pruning
- ☐ Code Container With Most Water
- ☐ Code Trapping Rain Water $O(1)$ space
- ☐ Prove why Two Pointers works in $O(N)$

MATHEMATICAL PROOF — $O(N)$ TWO SUM II

Claim: Inward movement never skips a valid target pair (i, j) .

Proof: Suppose $nums[L] + nums[R] > target$. Because array is sorted, for any $k > L$, $nums[k] + nums[R] \geq nums[L] + nums[R] > target$. Thus, index R CANNOT pair with ANY element to the right of L to form target. It is safe to decrement R !

GOLDEN RULES — NEVER FORGET

Rule 1 — Always Sort for Multi-Sum

For 3Sum / 4Sum, sort first! This enables duplicate skipping and $O(N)$ two-pointer convergence.

Rule 2 — Move the Shorter Wall

In Container With Most Water & Trapping Rain Water, the shorter height is ALWAYS the bottleneck!